BillLedger

Multi-Tenant SaaS Invoice & Billing System

Built with Flask, SQLAlchemy and Jinja2

Project Report

Prepared by: Muhammad Faizan Farooq

November 2026

1. Abstract

BillLedger is a multi-tenant Software-as-a-Service (SaaS) application for invoicing and billing. Many companies use one deployed application, and each company works inside its own isolated workspace. Within a workspace a company manages clients, creates invoices with tax, records payments, emails PDF invoices, and automates recurring billing and payment reminders. Access is controlled through three roles: Admin, Accountant and Client. The system is built with Flask, SQLAlchemy and Jinja2, and was developed in stages: a static frontend prototype first, then the Flask backend.

2. Problem statement and objectives

Small businesses, agencies and freelancers often create invoices in spreadsheets or word processors. This leads to duplicate invoice numbers, unclear payment status, forgotten follow-ups and no single view of money owed. The objectives of this project are:

- Give each company a private workspace so data is never mixed between companies.
- Produce consistent, numbered invoices with tax, and deliver them as PDF by email.
- Track payment status automatically, including partial payments and overdue invoices.
- Automate repetitive work: recurring invoices and payment reminders.
- Control who can see and do what through role-based access.

3. System features

Module	What it does	Used by
Registration and login	Creates a company and its first admin in one step; secure login and logout.	Everyone
Dashboard	Billed, paid, awaiting payment and overdue totals with recent invoices.	Admin, Accountant
Clients	Add, edit and delete clients; outstanding balance per client.	Admin, Accountant
Invoices	Line items, tax, notes, automatic numbering, search and status filters, draft to sent to paid.	Admin, Accountant
Payments	Record full or partial payments; status updates automatically; overpayment is blocked.	Admin, Accountant
PDF and email	Download an invoice as PDF and email it to the client with the PDF attached.	Admin, Accountant
Recurring billing	Monthly or yearly plans that create and email invoices on schedule.	Admin, Accountant
Reminders	Emails 3 days before the due date, on the due date, and every 7 days while overdue.	Automatic
Team management	Add team members and change roles.	Admin
Client portal	Clients view and download their own invoices, read-only.	Client

4. Technology stack

Layer	Technology
Backend	Python, Flask, Blueprints, application factory pattern
Database	SQLAlchemy ORM, Flask-Migrate (Alembic); SQLite in development, PostgreSQL ready
Security	Flask-Login sessions, Flask-WTF CSRF protection and forms, Werkzeug password hashing
Frontend	Jinja2 templates, HTML5, CSS3 (responsive, light and dark mode), vanilla JavaScript
PDF and email	xhtml2pdf for PDF generation, Flask-Mail over SMTP

5. System architecture

The browser sends requests to a Flask application built with an application factory and two blueprints: **auth** (register, login, logout) and **main** (dashboard, clients, invoices, payments, team, recurring billing and the client portal). Service modules handle PDF generation, email and the daily job. All data access goes through SQLAlchemy. A scheduler (Windows Task Scheduler or cron) runs one command each day to create recurring invoices and send reminders.

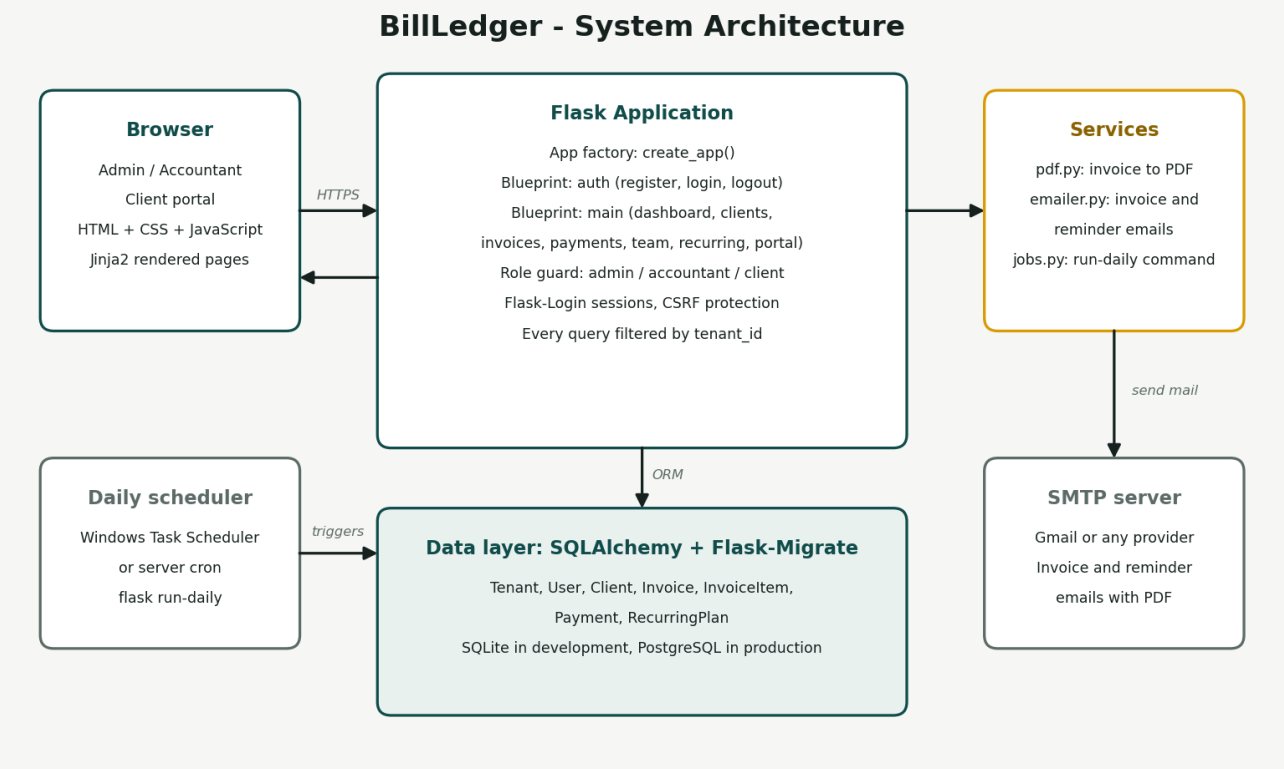


Figure 1: BillLedger system architecture

6. Database design

The database has seven tables. Every business table stores a **tenant_id**, which is the foundation of multi-tenant isolation.

Table	Key fields	Purpose
tenants	name, email, currency, default_tax_rate	One row per company workspace
users	tenant_id, email, password_hash, role, client_id	Admins, accountants and client logins
clients	tenant_id, name, email, phone, address	Customers who receive invoices
invoices	tenant_id, client_id, number, issue_date, due_date, status, tax_rate, last_reminded	Invoice header; number is unique per tenant
invoice_items	invoice_id, description, quantity, unit_price	Invoice lines
payments	tenant_id, invoice_id, amount, method, paid_on, reference	Payments received
recurring_plans	tenant_id, client_id, description, amount, frequency, next_run, active	Automatic billing schedules

Totals such as subtotal, tax, total, paid amount and balance are calculated from the line items and payments, so they can never drift out of sync with the stored data.

7. Security and multi-tenant isolation

- **Tenant filter:** every query is restricted to the logged-in user's tenant. Opening another company's record returns 404.
- **Role guard:** a decorator limits each route to allowed roles. A Client opening a staff page receives 403.
- **Client scope:** client users only see invoices linked to their own client record, and never drafts.
- **Passwords and forms:** passwords are stored as hashes; all forms use CSRF tokens; login redirects accept local paths only.
- **Secrets:** keys and mail credentials live in a .env file that is never committed to Git.

Isolation is enforced in the application layer. For stricter guarantees at scale, PostgreSQL row-level security can be added.

8. Invoice workflow

- An invoice starts as **draft** and can be edited or deleted. Marking it as sent, or emailing it, changes the status to **sent**.
- Payments can only be recorded on sent invoices. When the balance reaches zero the status becomes **paid**.
- If the due date passes with a balance remaining, the status becomes **overdue** automatically.
- Recurring plans create a new invoice on each run date, due 14 days later, and email it to the client.

- Reminders are emailed 3 days before the due date, on the due date, and every 7 days while overdue, at most once per day.

9. User interface

The interface uses a consistent teal and amber theme with a sidebar, responsive layout, and light and dark mode. The screenshot below shows the dashboard and the invoice form.

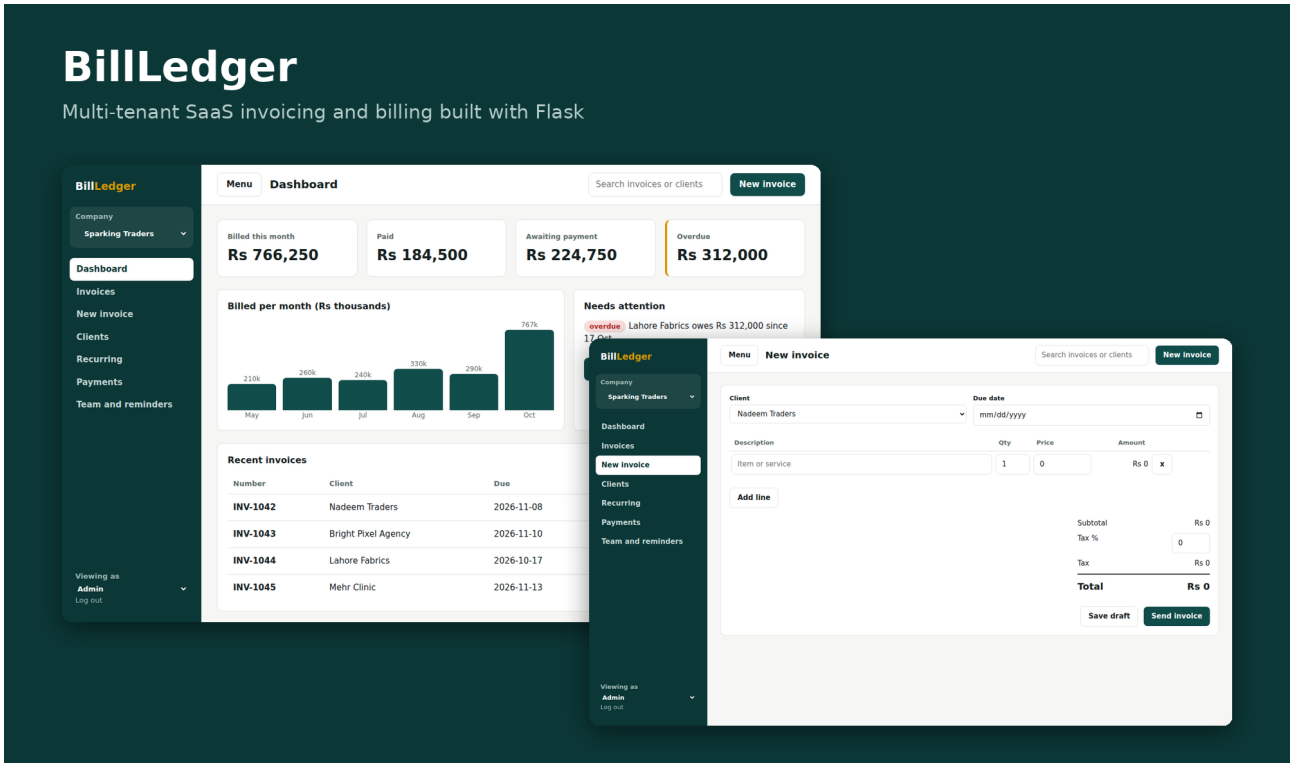


Figure 2: Dashboard and new invoice screens

10. Business benefits

- **Faster cash collection:** invoices go out immediately by email, and automatic reminders reduce late payments.
- **Less manual work:** recurring billing removes the monthly task of recreating the same invoices.
- **Fewer errors:** automatic numbering and calculated totals prevent duplicate numbers and arithmetic mistakes.
- **Clear financial view:** the dashboard shows what is billed, paid, pending and overdue at any moment.
- **Professional image:** consistent PDF invoices and a client portal improve how customers see the business.
- **Safe team work:** roles let owners share work with accountants without exposing everything.
- **Scalable model:** one deployment serves many companies, which suits a subscription (SaaS) business.
- **Low running cost:** an open-source stack and SQLite or PostgreSQL keep hosting and licence costs small.

11. Business disadvantages and limitations

- **No online payments yet:** payments are recorded manually, so clients cannot pay by card from the invoice.
- **Email depends on setup:** reminders and invoice emails work only after SMTP is configured, and delivery can land in spam.
- **Scheduler is external:** the daily job needs a cron job or Task Scheduler, and a missed day can miss a reminder date.
- **Basic accounting only:** there is no general ledger, expense tracking, inventory, or tax-authority integration.
- **Single currency display and no multi-language support**, which limits international use.
- **No password reset or email invitations yet:** admins must share temporary passwords with team members.
- **Application-level isolation:** a coding mistake in a query could expose data, so automated tests and reviews are important.
- **Not yet production hardened:** no automated tests, rate limiting, audit log or backups are included in this version.
- **Compliance work remains:** invoice formats and tax rules differ by country and must be reviewed before commercial use.

12. Future improvements

- Online payments with Stripe or Paymob and automatic payment confirmation.
- Automated tests with pytest, continuous integration, and Docker deployment.
- Password reset, email verification and email-based team invitations.
- Audit log, CSV and Excel export, and dashboard charts from live data.
- Multi-currency support, invoice templates with company logo, and expense tracking.

13. Conclusion

BillLedger delivers the core of an invoicing SaaS: isolated company workspaces, a complete invoice-to-payment workflow, PDF and email delivery, automation for recurring billing and reminders, and role-based access including a client portal. The modular Flask structure makes it straightforward to add online payments, testing and deployment next, and the project demonstrates practical full-stack skills in database design, security, background automation and user interface design.